

Technical Report

Table of Contents

简易步兵机器人电控系统技术文档

1. 项目目标

本项目基于 STM32 HAL 工程思想，实现一个简易步兵机器人电控系统的软件框架。系统覆盖底盘控制、云台控制、按键输入、PID 闭环、安全保护和任务调度，目标是形成一个可继续接入真实电机、编码器和 IMU 的工程基础。

2. 功能范围

已实现：

- 主控任务调度
- 遥控按键输入
- STOP/NORMAL 整车模式
- 独立、小陀螺、跟随云台三种底盘模式
- X 型四轮全向底盘运动学
- 四路底盘电机速度 PID
- yaw/pitch 云台目标角控制
- pitch 角度限位
- 云台角度 PID
- 电机输出限幅
- 急停保护
- L1 呼吸灯、L2 模式灯接口
- 串口数据包解析框架
- C620/M3508 CAN 电调协议打包和反馈解析

- 独立安全状态机

待接入/待实车验证：

- 具体 STM32 型号和引脚配置
- 真实 PWM/CAN 电机输出
- 编码器速度换算
- IMU yaw/pitch 角度读取
- 实车 PID 参数整定
- 具体 CAN 外设过滤器、发送邮箱和接收中断配置

3. 软件架构

App

- ├─ Remote: 按键和串口输入, 输出 remote_cmd_t
- ├─ Chassis: 底盘模式、运动学、轮速 PID
- ├─ Gimbal: 云台目标角、角度 PID、Pitch 限位
- ├─ PID: 通用闭环控制器
- ├─ Safety: 急停、遥控丢失、限幅等安全状态
- ├─ C620 CAN: RM 常见电调控制帧和反馈帧解析
- └─ BSP Port: STM32 HAL 硬件适配层

架构设计原则：

- main.c 不放控制逻辑，只调用 App_Init() 和 App_Loop()。
- remote.c 只产生命令，不直接控制电机。
- chassis.c 只关心底盘速度和轮速输出。
- gimbal.c 只关心目标角、反馈角和云台输出。
- bsp_port.c 统一对接 GPIO、TIM、CAN、编码器、IMU。
- safety.c 只做安全状态汇总，不直接写电机。
- c620_can.c 只做协议打包和解析，不直接调用 HAL CAN。

4. 数据流

主控制周期为 `ROBOT_CONTROL_PERIOD_MS = 5 ms`。

Remote_Update()

-> 生成 remote_cmd_t

Gimbal_Control()

-> 根据 key2/key3 更新目标角

-> 读取 IMU yaw/pitch

-> PID 输出云台电机控制量

Chassis_Control()

-> 根据底盘模式修正 vx/vy/wz

-> X 型全向轮解算

-> 读取编码器速度

-> PID 输出底盘电机控制量

5. 底盘控制

底盘输入为车体坐标系速度：

- vx：前后速度
- vy：左右速度
- wz：旋转角速度

X 型四轮全向轮速度分配：

$$w1 = +vx - vy - wz * K$$

$$w2 = +vx + vy + wz * K$$

$$w3 = -vx + vy - wz * K$$

$$w4 = -vx - vy + wz * K$$

其中 K 为底盘几何系数，实车时需要根据轮距和安装尺寸标定。

6. 云台控制

云台使用目标角闭环：

target_angle - imu_angle -> PID -> motor_output

Yaw：

- key2 短按增加目标角。
- key2 长按连续增加目标角。

- yaw 误差使用 Robot_WrapAngleDeg() 限制到 $[-180, 180]$ 。

Pitch:

- key3 短按增加目标角。
- key3 长按连续增加目标角。
- 目标角限制在 $[-20, 30]$ 。

7. 安全保护

安全保护优先级高于功能完整度:

- 急停: 进入 ROBOT_STOP 后底盘和云台输出全部清零。
- 输出限幅: 所有电机输出限制在安全范围内。
- Pitch 限位: 从目标角源头限制, 避免云台撞机械限位。
- 遥控超时接口: 若后续接真实遥控器, 应让 valid 和 last_update_ms 反映真实链路状态, 并在超时后强制停机。

8. 硬件适配说明

所有硬件相关函数集中在 bsp_port.c:

```
uint32_t BSP_GetTickMs(void);
uint8_t BSP_ReadKey(key_id_t key);
void BSP_SetMotorOutput(motor_id_t motor, float output);
float BSP_GetMotorSpeed(motor_id_t motor);
float BSP_GetYawAngleDeg(void);
float BSP_GetPitchAngleDeg(void);
void BSP_SetLedL1(float duty_0_to_1);
void BSP_SetLedL2(uint8_t on);
```

移植到真实工程时, 只需要替换这些函数内部实现, 不需要改控制层逻辑。

9. 完成度说明模板

当前版本完成了软件框架、状态机、运动学、PID 接口和安全保护。由于不同开发板引脚、电机驱动和 IMU 型号不同, 硬件适配层需要根据实际设备补充。若已经接入实车, 应在测试记录中补充电机悬空测试、编码器反馈测试和云台角度响应测试。

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:15